RISC-V 并行加速系统

摘要

针对边缘智能感知、工业实时控制与数字信号处理场景中通用微控制器算力密度不足、软件执行矩阵向量乘加（GEMV/MAC）开销大的痛点，本作品基于紫光同创 Logos-2 系列 FPGA（`PG2L200H-6FBB676`），采用纯 Verilog-2001 硬件描述语言从零构建了一套软硬件协同的 **“RISC-V 并行加速系统”**。

在通用处理器内核层面，系统实现了支持 **RV32IM** 指令集的经典 5 级流水线（IF-ID-EX-MEM-WB）处理器。针对流水线数据冒险与结构冒险，设计了全通路数据前推网络（EX-EX、MEM-EX 及寄存器堆内部写穿）、Load-Use 与 SW→LW 结构停顿控制器以及停顿期间的前推操作数同步锁存机制；针对软件模拟乘除法与控制跳转的性能瓶颈，在执行级集成了 2 拍硬件乘法与 32 拍恢复余数除法单元（RV32M），并在取指级构建了 **64 项直接映射分支目标缓冲（BTB）与 2-bit 饱和计数器分支历史表（BHT）动态分支预测器**，实现预测命中零气泡跳转。

在片上系统（SoC）与并行加速层面，系统以 **Wishbone B4** 1 主 4 从同步互联矩阵为骨干，集成 64 KB 双端口片上 SRAM、带 8 字节双 FIFO 的 UART0 控制器、64 位防撕裂 CLINT 定时器以及 **4 路并行 SIMD INT8 乘累加硬件加速器（`wb_mac_acc`）**。该加速器创新采用 32 位总线打包传输、**隐式写触发（Trigger-on-Write）**、偏置预置与**零延迟硬件 ReLU 激活**架构，彻底消除了传统 MMIO 加速器的控制握手开销。

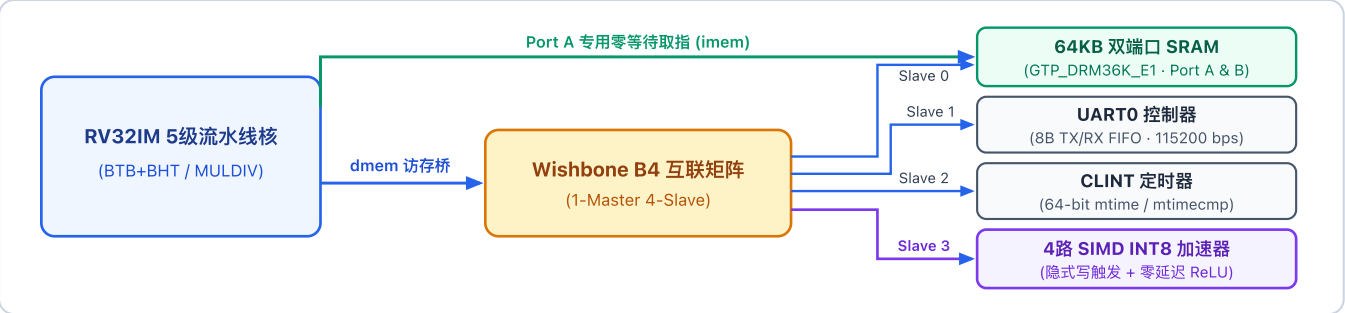
系统通过了 46 项 RISC-V 官方指令集测试并与黄金模型 Spike 实现 100% 逐指令提交差分吻合。在紫光同创远程 FPGA 实验平台上（27 MHz 板载晶振，静态时序最高工作频率 F_{max} = 58.03 MHz），整机仅耗用 3,359 个 LUT（占芯片资源 2.10%），EEMBC CoreMark 实测性能由纯 RV32I 基线的 `0.958` 跃升至 **`3.025 CoreMark/MHz`**（提升 **215.8%**）；在 16 × 64 INT8 全连接层基准测试中，并行加速器相对软件移位加实现 **`24.98×` 加速**，相对 RV32IM 标量硬件乘法实现 **`3.32×` 加速**，验证了国产 FPGA 软硬件协同加速架构的高能效与高可靠性。

第一部分 作品概述

1.1 功能与特性

本作品构建了一个集通用计算、实时控制与算子并行加速于一体的完整 RISC-V 片上系统（SoC），核心功能与特性如下：

1. **RV32IM 五级流水线通用处理**：完整支持 RV32I 基础整数指令集（37 条）与 RV32M 硬件乘除法扩展（8 条），具备全通路数据前推、硬件冒险停顿及 BTB+BHT 动态分支预测功能，可直接运行无操作系统依赖的裸机 C 语言程序与权威基准测试。
2. **标准总线与完备片上外设**：基于 Wishbone B4 总线挂载 64 KB 双端口 SRAM（支持取指与访存并行无冲突访问）、115200 bps 硬件流控 UART 串口、64 位高精度 CLINT 定时器及板级 LED/按键状态监控接口。
3. **边缘 AI 算子硬件并行加速**：内置 4 路 SIMD INT8 乘累加与硬件 ReLU 激活加速单元，支持向量点积、全连接层（GEMV）与卷积核的流式并行加速。
4. **全链路自动化验证与板级自检**：支持官方指令集回归、Spike 逐周期差分比对，以及上电自动执行算子加速对比与 > 10 s CoreMark 巡检报告输出。



1.2 应用领域

本系统以"高主频能效比通用 RISC-V 控制核 + 低开销定制并行算子单元"的紧凑异构架构，广泛适用于对体积、功耗及实时响应敏感的国产化边缘嵌入式场景：

1. **智能工业传感与预测性维护**：在工业电机振动监测、轴承故障诊断及智能声学检测中，CPU 负责传感器采集调度与通信协议栈，SIMD 加速器实时完成时频特征提取（FIR 滤波、FFT 蝶形乘加）与轻量级多层感知机（MLP）分类推理。
2. **低功耗边缘智能视觉与语音终端**：在关键词唤醒（KWS）、手写数字/字符识别及轻量级目标检测终端中，承担量化神经网络（INT8 TFLite Micro 模型）中占算力 80% 以上的全连接层与逐点卷积加速。
3. **国产自主可控教学与原型验证平台**：全系统采用零厂商原语锁定的标准 Verilog-2001 编写，配套完备的 Freestanding C SDK 与 Spike 差分验证框架，可直接作为国产 FPGA 软硬件协同设计与微架构实验平台。

1.3 主要技术特点

1. **纯净 RTL 设计与高可移植性**：CPU 内核、总线与外设完全采用标准 Verilog-2001 行为级描述，无厂商黑盒 IP 锁定，在 Verilator `-Wall` 严苛静态检查下保持 **0 警告**，同时可被紫光 PDS 自动推断映射为底层 `GTP_DRM36K_E1` 块 RAM 与 `GTP_APM_E2` DSP 硬核。
2. **BRAM 同步时序折叠与冒险闭环**：通过将取指读地址（IF 级）与 Load 访存读地址（EX 级）提前一拍发出，将 FPGA 同步 BRAM 的 1 拍固有读延迟无缝折叠进 5 级流水线；配合停顿期间前推值同步锁存机制，彻底消除前推源退休导致的数据撕裂隐患。
3. **极简高效的加速器微架构**：摒弃繁琐的"写操作数→写控制位→轮询完成位→读结果"握手流程，采用**写操作数 B 即当拍触发 4 路有符号乘累加**的隐式触发机制，结合偏置可写预载与组合逻辑 ReLU 读端口，将每 4 次 MAC 运算压缩至仅需 2 条 `sw` 指令。

1.4 主要性能指标

评估维度	实测性能指标	备注说明
执行效率	**`3.025 CoreMark/MHz`**	较纯 RV32I 基线（`0.958`）提升 **+215.8%**
工作频率	板级 **27.00 MHz** / F_{max} **58.03 MHz**	27 MHz 下 WNS `+20.228 ns`，0 时序违例
逻辑资源	**3,359 LUTs (2.10%)** / 5,067 FFs	另含 33 个 DRM 块 RAM、8 个 APM 乘法器
资源效率	**0.901 CoreMark/MHz / 1k LUTs**	58.03 MHz 下折合 **52.26 CoreMark / 1k LUTs**
算子加速	**24.98×** (vs 软乘) / **3.32×** (vs 硬件 MUL)	16 × 64 INT8 全连接层（`2,537` 周期）

1.5 主要创新点

1. **隐式写触发与算子融合 SIMD 架构**：首创单写触发（`ACC_OPB_TRIG`）+ 偏置预载 + 零延迟 ReLU 读端口的 Wishbone 从设备加速器，仅用 134 行 RTL 与 4 个 APM 硬核实现 3.32×~24.98× 算子提速。

2. **Taken-Only 分配与防重复训练动态分支预测**：设计 64 项 BTB + 2-bit BHT 预测器，引入停顿门控训练机制，使 CoreMark 计时周期缩减 16.3%。
3. **周期精确（Cycle-Exact）软硬件验证闭环**：建立 Spike 逐指令差分验证与板级硬件性能计数体系，实现紫光 FPGA 实测周期与仿真 **单拍不差（660,991 ticks）**。

1.6 设计流程

本项目采用**“规格先行 → 模块单测 → Spike 差分验证 → 系统基准剖析 → 国产 FPGA 综合上板”**的五步闭环工程流程。先编写各模块微架构 Spec，完成 Verilog 实现与 Verilator 零警告检查；再通过 12 项单元测试、46 项官方 ISA 测试及 Spike 逐指令比对确认功能正确；随后移植 C 运行时与 CoreMark/算子基准完成软硬件协同调优；最终经 PDS 2022.2 综合签核并部署至紫光远程实验箱实测。



第二部分 系统组成及功能说明

2.1 整体介绍

本系统由 **五级流水线处理器核心（`core`）**、**Wishbone B4 片上互联矩阵（`wb_interconnect`）**、**存储与外设子系统（`wb_sram`、`wb_uart`、`wb_clint`）**、**并行算子加速器（`wb_mac_acc`）** 以及 **板级复位与监控顶层（`fpga_top`）** 五大部分构成。系统整体架构框图如下：

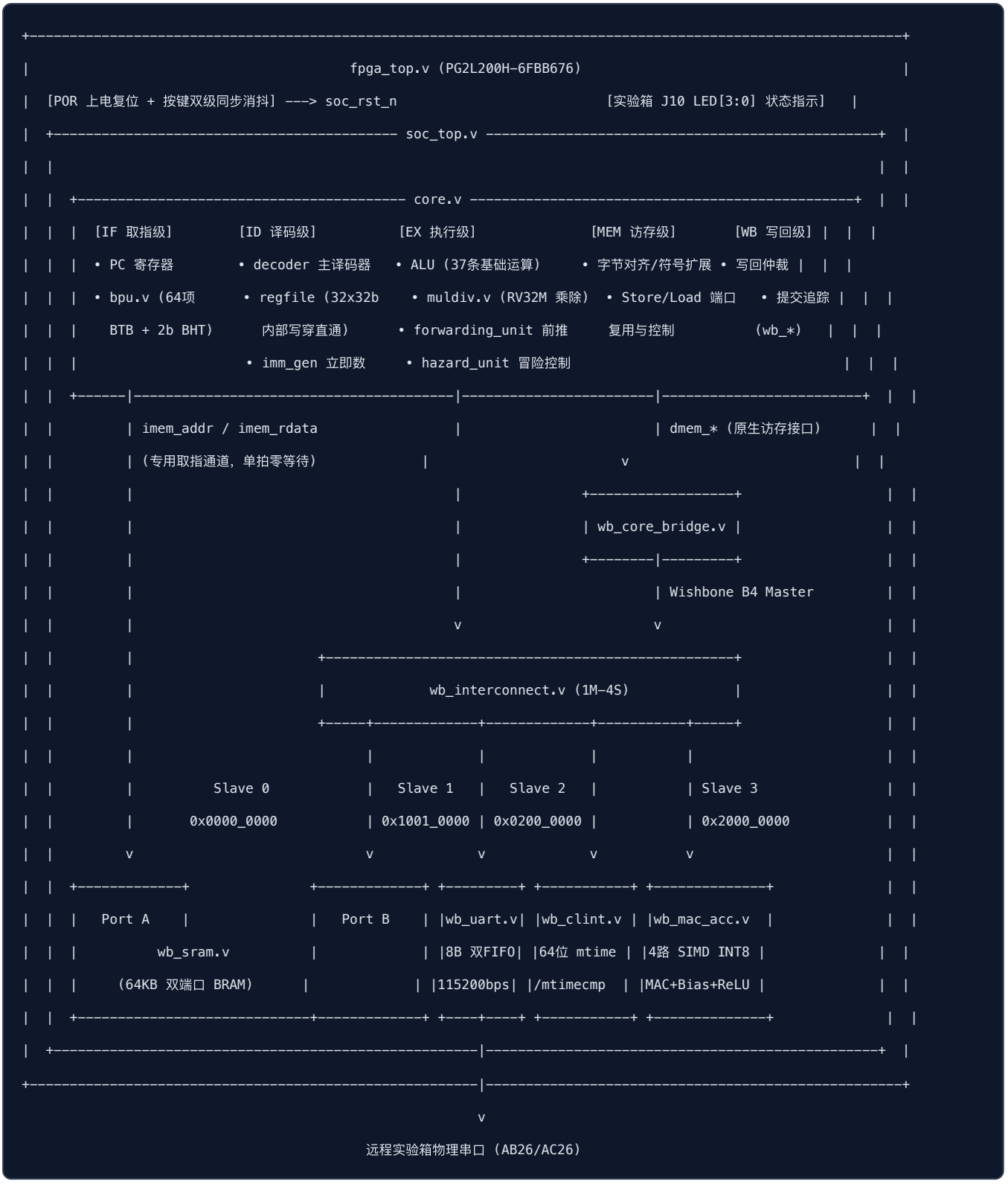

fpga_top.v (PG2L200H-6FBB676)
[POR 上电复位 + 按键双级同步消抖] ---> soc_rst_n
[实验箱 J10 LED[3:0] 状态指示]
soc_top.v
core.v
[IF 取指级]
• PC 寄存器
• bpu.v (64项 BTB + 2b BHT)
[ID 译码级]
• decoder 主译码器
• regfile (32x32b 内部写穿直通)
• imm_gen 立即数
[EX 执行级]
• ALU (37条基础运算)
• muldiv.v (RV32M 乘除)
• forwarding_unit 前推
[MEM 访存级]
• 字节对齐/符号扩展
• Store/Load 端口复用与控制
[WB 写回级]
• 写回仲裁
• 提交追踪 (wb_*)
• hazard_unit 冒险控制
imem_addr / imem_rdata
(专用取指通道，单拍零等待)
dmem_* (原生访存接口)
wb_core_bridge.v
Wishbone B4 Master
wb_interconnect.v (1M-4S)
Slave 0
0x0000_0000
Slave 1
0x1001_0000
Slave 2
0x0200_0000
Slave 3
0x2000_0000
Port A
wb_sram.v
(64KB 双端口 BRAM)
Port B
wb_uart.v
115200bps
8B 双FIFO
wb_clint.v
64位 mtime
/mtimecmp
wb_mac_acc.v
4路 SIMD INT8
MAC+Bias+ReLU
远程实验箱物理串口 (AB26/AC26)



图 1：SoC 双通路拓扑与 Wishbone B4 单拍总线交互架构图

各子模块协同关系与地址空间规划：

- **哈佛/冯·诺依曼混合取指访存架构**：利用紫光同创 FPGA 双端口块 RAM（`GTP_DRM36K_E1`）的物理特性，`wb_sram` 的 Port A 直接与 CPU 的取指级（IF）相连，提供零总线仲裁延迟的单周期取指；Port B 则作为 Slave 0 挂载在 Wishbone B4 数据总线上，供 Load/Store 指令读写数据。代码与数据共享 `0x0000_0000 ~ 0x0000_FFFF`（64 KB）统一编址空间，既保证了取指与数据访存的并行吞吐，又支持 C 语言的统一指针寻址。
- **系统地址映射表**：

起始地址	结束地址	容量	挂载外设模块	访问属性	功能描述
`0x0000_0000`	`0x0000_FFFF`	64 KB	片上双端口 SRAM (`wb_sram`)	R/W/X	存放指令代码段、只读数据段、全局数据段与运行栈
`0x0200_0000`	`0x0200_FFFF`	64 KB	核心本地中断器 (`wb_clint`)	R/W	RISC-V 标准 64 位计时器 `mtime` (`0xBFF8`) 与比较寄存器 `mtimecmp` (`0x4000`)
`0x1001_0000`	`0x1001_001F`	32 B	串口控制器 (`wb_uart`)	R/W	兼容 SiFive FE310 寄存器接口，含 8 字节收发双 FIFO 与满/空状态标志
`0x2000_0000`	`0x2000_001F`	32 B	并行算子加速器 (`wb_mac_acc`)	R/W	4 路并行 INT8 乘累加、偏置预置、隐式写触发及零延迟 ReLU 激活读端口
`0xFFFF_FFF0`	`0xFFFF_FFFF`	16 B	状态控制信箱 (`SIM_CTRL`)	W	报告程序自检 PASS/FAIL 状态，驱动板级 LED1 (`U15`) 锁定指示与仿真退出

2.2 硬件系统介绍

2.2.1 硬件整体介绍

本系统目标硬件平台为紫光同创远程 FPGA 实验平台上的 **盘古 200K-MINI 核心板 + 实验箱扩展底板**。核心器件为紫光同创 Logos-2 系列 **`PG2L200H-6FBB676`**（Speed Grade `-6`，FBGA676 封装），片上拥有 159,800 个 6 输入查找表（LUT）、319,600 个触发器（FF）、415 个 36Kb 双端口块存储器（DRM）以及 280 个算术处理单元（APM DSP 硬核）。

板级硬件输入输出接口分配如下：

- **系统时钟（`sys_clk`）**：由核心板 27.00 MHz 有源晶振提供，接入专用全局时钟引脚 `D18`（3.3V LVCMOS33，周期 `37.037 ns`）。
- **复位与按键输入（`rst_n`，`key[3:0]`）**：核心板复位键接入 `C22`，实验箱 J10 扩展面板 4 颗按键接入 `U21, W24, V24, AA23`。
- **状态指示灯（`led[3:0]`）**：接入实验箱 J10 扩展面板的 4 颗低电平点亮（Active-Low）单色 LED（引脚 `U16, U15, V17, V16`），分别实时映射硬件时钟心跳、C 程序/CoreMark 校验 PASS 锁定、每轮测试循环翻转以及 UART 发送活动状态。
- **物理串口通信（`uart_txd`，`uart_rxd`）**：通过实验箱仪器 FPGA 桥接通道连接被测芯片的 `AB26`（TX）与 `AC26`（RX），工作于 `115200 bps, 8N1` 模式，直通远程平台 Web 串口终端。

2.2.2 机械设计介绍

本作品属于 FPGA 数字芯片微架构与片上系统（SoC）设计课题，依托紫光同创盘古 `PG2L200H` 标准开发平台与远程实验箱机架运行，不涉及额外定制机械结构件。

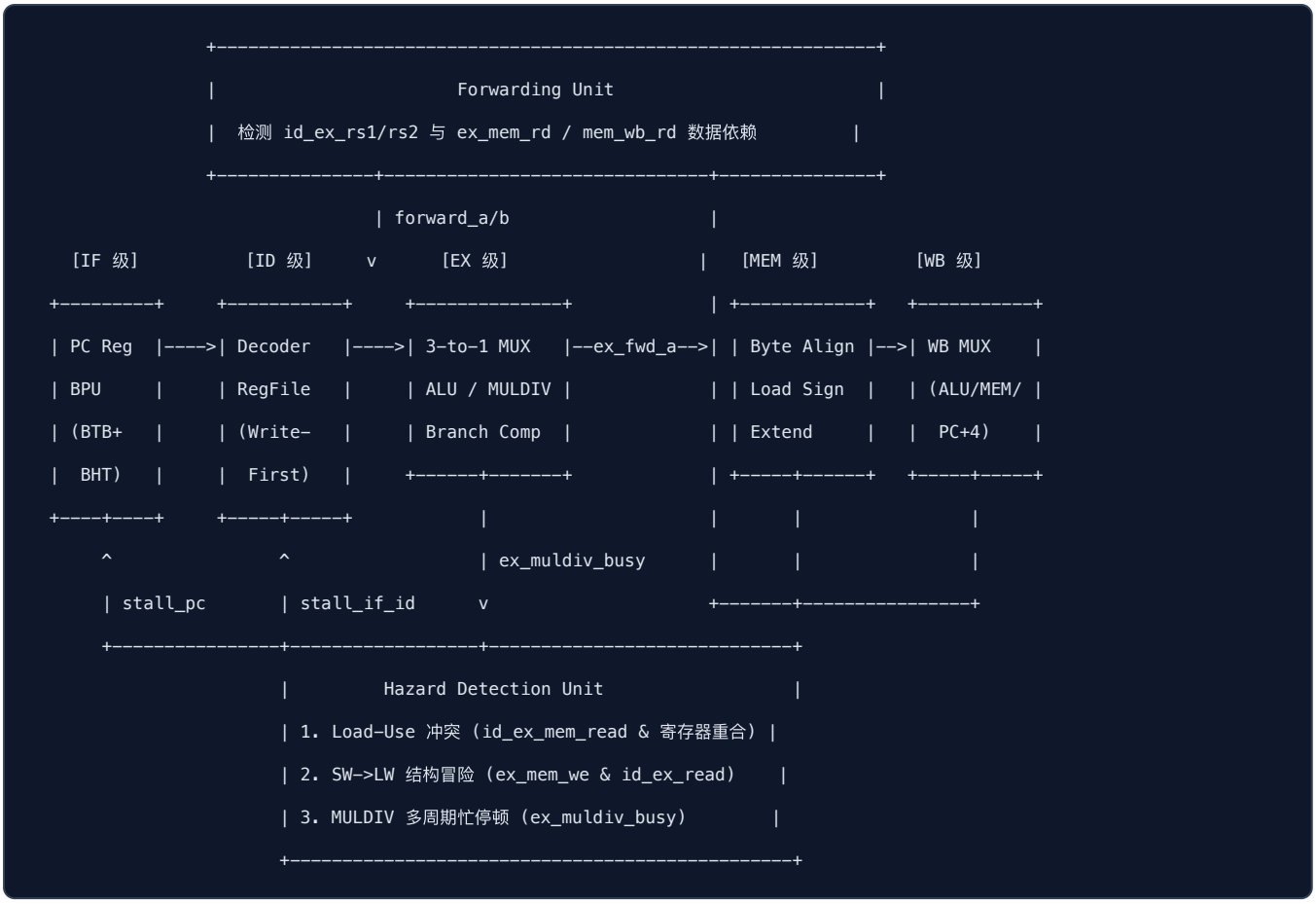
2.2.3 电路各模块介绍

以下从处理器顶层数据通路到各核心子模块，逐级给出具体逻辑设计图、关键输入/输出信号线及核心时序机制：

（1）五级流水线处理器核心（`core.v`）与数据前推/冒险控制（`forwarding_unit.v`，`hazard_unit.v`）

- **关键输入/输出信号线**：
  - **输入信号**：`clk`（系统时钟）、`rst_n`（同步低有效复位）、`imem_rdata[31:0]`（取指返回指令字）、`dmem_rdata[31:0]`（访存读回数据）、`dmem_ready`（Wishbone 总线应答就绪）。
  - **输出信号**：`imem_addr[31:0]`（取指字节地址）、`dmem_addr[31:0]` / `dmem_wdata[31:0]` / `dmem_we` / `dmem_re` / `dmem_wstrb[3:0]`（访存读写控制总线）、`wb_commit_valid` / `wb_commit_pc[31:0]` / `wb_commit_rd[4:0]` / `wb_commit_wdata[31:0]`（差分验证退休追踪总线）。

`core.v` 将指令执行划分为 **取指（IF）→ 译码（ID）→ 执行（EX）→ 访存（MEM）→ 写回（WB）** 五个流水级，各级之间由同步级间寄存器（`if_id_*`、`id_ex_*`、`ex_mem_*`、`mem_wb_*`）及随路有效位 `valid` 隔离：



- **全通路三级数据前推**：
  1. **EX-EX 前推（最高优先级，`forward = 2'b10`）**：当处于 MEM 级的上一条指令写寄存器（`ex_mem_reg_write && ex_mem_rd != 0`）且与当前 EX 级源寄存器重合时，直接将 `ex_mem_fwd_data`（ALU 结果或 `PC+4`）旁路至 EX 级输入；
  2. **MEM-EX 前推（次优先级，`forward = 2'b01`）**：当处于 WB 级的指令写寄存器且与 EX 级源寄存器重合时，将 `wb_wdata` 旁路至 EX 级输入；
  3. **RegFile 内部写穿（Write-First）**：在 `regfile.v` 内部，当 WB 级写使能有效且 `wb_rd == id_rs1/rs2` 时，直接组合旁路 `wb_wdata`，消除相隔 2 条指令的读写冲突。
- **停顿期间的前推操作数锁存机制（关键微架构保护）**：

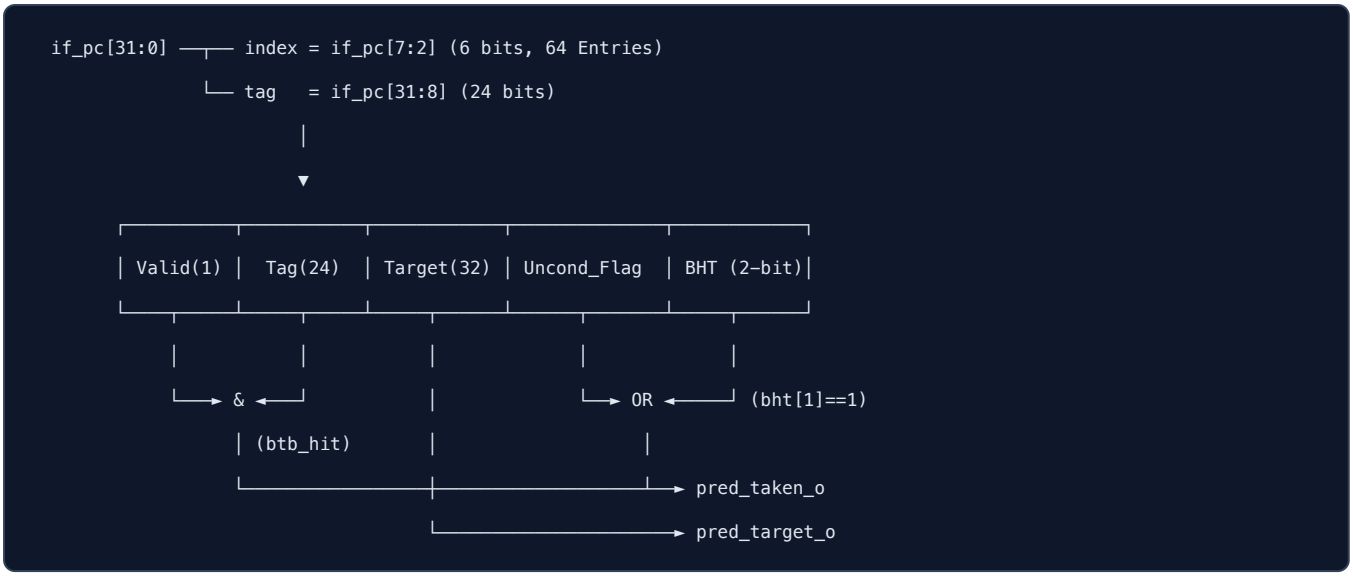
当 EX 级因乘除法迭代或结构冒险被冻结（`stall_id_ex = 1`），而 WB 级的生产者指令在本拍完成写回并退休离开流水线时，纯组合前推信号会在下一拍消失。为此，`core.v` 在 `stall_id_ex` 期间执行同步采样锁存：

```
end else if (stall_id_ex) begin
    id_ex_rs1_data <= ex_fwd_val_a; // 实时锁存当拍前推后的最新操作数 A
    id_ex_rs2_data <= ex_fwd_val_b; // 实时锁存当拍前推后的最新操作数 B
end
```

从而彻底杜绝了多周期停顿下的“前推源退休逃逸”隐患。

（2）动态分支预测器（`bpu.v`）

为了消除条件分支与跳转指令在 EX 级裁决带来的 2 拍冲刷惩罚，系统在 IF 级集成了 **64 项直接映射 BTB + 2-bit 饱和计数器 BHT**：



- **关键输入/输出信号线**：
  - **输入信号**：`if_pc_i[31:0]`（IF 级当前取指 PC）、`update_en_i`（EX 级分支训练使能，受 `!stall_id_ex` 门控）、`update_pc_i[31:0]`、`actual_taken_i`（真实跳转方向）、`actual_target_i[31:0]`（真实跳转目标）、`is_uncond_i`（JAL/JALR 标志）。
  - **输出信号**：`pred_taken_o`（当拍预测跳转有效）、`pred_target_o[31:0]`（当拍预测目标地址）。
- **IF 级单拍组合预测**：取指级利用 `if_pc[7:2]` 索引 64 项表项。当 `btb_valid && (btb_tag == if_pc[31:8])` 命中，且无条件跳转标志 `btb_uncond == 1` 或二位饱和计数器最高位 `bht[1] == 1`（弱跳 `2'b10` / 强跳 `2'b11`）时，当拍输出 `pred_taken_o = 1` 并直接将 `if_next_pc` 重定向至 `pred_target_o`，实现 **0 气泡跳转**。
- **EX 级精确纠偏与 Taken-Only 分配**：预测状态随 `if_id` → `id_ex` 级间寄存器前传至 EX 级，与真实分支计算结果比对。仅当方向预测错误（`ex_branch_taken != id_ex_pred_taken`）或目标地址不匹配时才触发 `ex_branch_mispred` 冲刷流水线并重定向。未命中 BTB 的分支仅在实际发生跳转（Taken）时才分配新表项（Taken-Only Allocation），极大提升了 64 项表项的有效容量利用率。

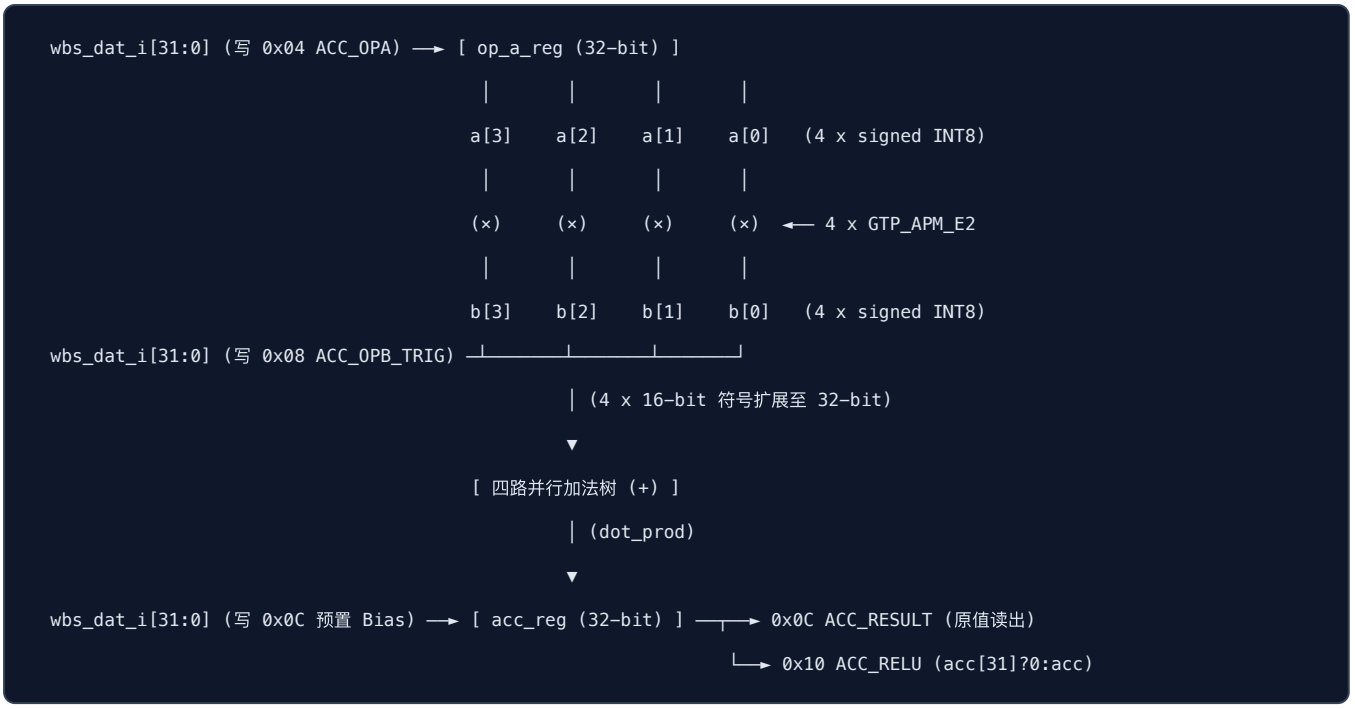
（3）RV32M 硬件乘除法单元（`muldiv.v`）

`muldiv.v` 位于 EX 级，完整支持 `MUL/MULH/MULHSU/MULHU` 与 `DIV/DIVU/REM/REMU` 共 8 条指令：

- **关键输入/输出信号线**：
  - **输入信号**：`valid_i`（EX 级 M 扩展指令有效）、`op_i[2:0]`（即 `funct3` 运算编码）、`rs1_data_i[31:0]`（被乘数/被除数）、`rs2_data_i[31:0]`（乘数/除数）。
  - **输出信号**：`busy_o`（多周期运算忙停顿请求，直连 `hazard_unit`）、`done_o`（运算完成脉冲）、`result_o[31:0]`（32 位积/商/余数输出）。
- **33×33 符号扩展硬核乘法通路**：根据指令有符号/无符号属性，将两个 32 位操作数最高位扩展为 33 位有符号数（有符号扩展 `op[31]`，无符号扩展 `1'b0`），经 1 拍流水寄存器输出 66 位乘积，自动综合映射为紫光 `GTP_APM_E2` DSP 硬核，乘法仅停顿 1 拍（总耗时 2 拍）。
- **32 拍恢复余数除法状态机与快速旁路**：对除以 0（商返 `-1`、余返被除数）及有符号溢出（`-2^31 / -1`）实现 **1 拍硬件快速旁路**；常规除法通过 64 位余数移位寄存器执行 32 拍恢复余数试商迭代，计算完成当拍拉高 `done_o` 释放 `hazard_unit` 停顿。

（4）4 路并行 SIMD INT8 乘累加硬件加速器（`wb_mac_acc.v`）

`wb_mac_acc.v` 作为 Slave 3 挂载于 Wishbone 总线（基址 `0x2000_0000`），其核心数据通路如下图所示：



- **关键输入/输出信号线与寄存器接口定义**：
  - **总线信号线**：输入 `wbs_cyc_i / wbs_stb_i / wbs_we_i / wbs_adr_i[31:0] / wbs_dat_i[31:0]`，输出 `wbs_ack_o`（单拍零等待应答）与 `wbs_dat_o[31:0]`（读回数据）。
  - `0x00 (ACC_CTRL)`：写 bit[0] 清零累加器 `acc_reg` 与批次计数器 `count_reg`；
  - `0x04 (ACC_OPA)`：写入打包的 4 个有符号 INT8 权重 `{w3, w2, w1, w0}` 并锁存；
  - `0x08 (ACC_OPB_TRIG)`：**隐式写触发端口**，写入打包的 4 个有符号 INT8 输入 `{x3, x2, x1, x0}` 的**同一时钟周期**，硬件立即并行完成 $\sum_{i=0..3} \text{sext}(w_i \times x_i)$ 并累加至 `acc_reg`；
  - `0x0C (ACC_RESULT)`：读出 32 位有符号累加结果，写入时直接将 `acc_reg` 预置为当前神经元偏置 `Bias`；
  - `0x10 (ACC_RELU)`：**硬件零延迟 ReLU 激活读端口**，组合逻辑返回 `(acc_reg[31] == 1'b1) ? 32'd0 : acc_reg`。

（5）Wishbone B4 总线、双端口 SRAM、UART0 与 CLINT 外设

- **关键输入/输出信号线**：主设备侧输入 `wbm_adr_i / wbm_dat_i / wbm_we_i / wbm_sel_i / wbm_stb_i / wbm_cyc_i`，输出 `wbm_dat_o / wbm_ack_o`；物理外设侧输出串口信号 `uart_tx_o`、输入 `uart_rx_i` 以及调试状态 `sim_Finish_o / sim_err_code_o[31:0]`。
- **`wb_core_bridge.v` & `wb_interconnect.v`**：将 CPU 访存信号转为标准 Wishbone B4 Classic 单拍事务；互联器在请求拍根据高 16 位地址组合路由 `stb_o`，并在响应拍根据锁存的 `sel_r` 路由从设备 `dat_o/ack_o`，对未映射地址返回单拍虚拟 `ACK=1`，防止总线死锁。
- **`wb_sram.v`**：16384 × 32-bit（64 KB）可推断双端口 BRAM，支持字节写掩码 `wbs_sel_i[3:0]` 与 `$readmemh("firmware_words.hex", mem)` 比特流预加载。
- **`wb_uart.v`（`uart_tx.v`，`uart_rx.v`）**：内置 8 字节深度的同步 TX FIFO 与 RX FIFO，解耦 CPU 突发连续 `sw` 写字符与物理串口串行发送速率差；波特率分频器在 27 MHz 下配置 `CLKS_PER_BIT = 234`（对应 115200 bps，误差仅 0.16%）。

2.3 软件系统介绍

2.3.1 软件整体介绍

本系统软件栈采用**零外部操作系统依赖的 Freestanding 裸机架构**，配合宿主机端 Python 自动化验证与固件构建脚本，构成"编译 → 差分仿真 → 比特流镜像生成 → 远程平台串口交互"全流程闭环：

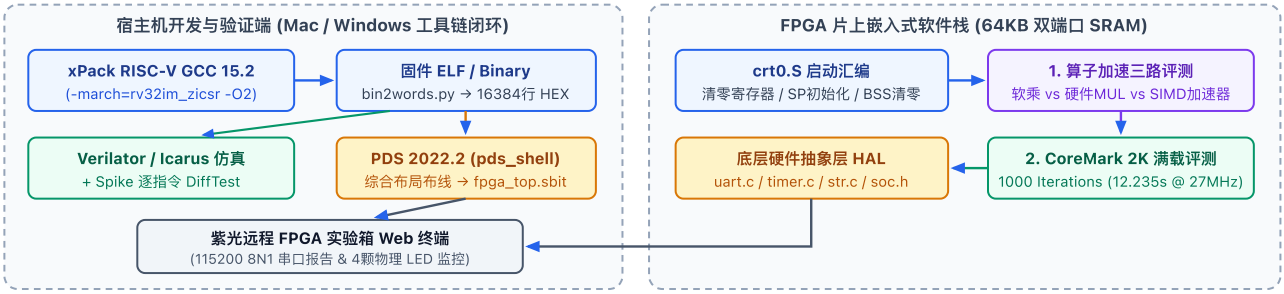




图 2：宿主机一键自动化回归测试（make lint / isa-test / difftest / test-acc）闭环验证结果

2.3.2 软件各模块介绍

以下按照**从顶层应用到底层硬件抽象（Top-to-Bottom）**的调用关系，逐级给出各软件模块核心函数的执行流程图及关键输入、输出变量说明：



（1）顶层应用入口与自动巡检调度模块（`main()` & `sw/common/crt0.S`）

- **函数执行流程**：上电复位进入 `_start` → 清零 `x1 ~ x31` 通用寄存器 → 初始化全局指针 `gp` 与栈指针 `sp = 0x0001_0000` → 循环按字清零 `[__bss_start, __bss_end)` → 跳转顶层 `main()` 依次调用算子加速评测与 CoreMark 评测 → 将 `main()` 返回码写入 `SIM_CTRL` 寄存器 → 在 `FPGA_BOARD_LOOP` 模式下延时约 1.5 秒后跳回 `_start` 开启下一轮自动巡检。
- **关键输入/输出变量**：

- **输入变量**：链接脚本导出的段边界符号 `__bss_start`、`__bss_end`、`__stack_top`；
- **输出变量**：`int rc`（`main()` 返回状态码，`0` 代表全部测试通过），映射写至 `SIM_CTRL_REG (0xFFFF_FFF0)`（写 `1` 点亮板载 `LED1` 并翻转 `LED2`；写 `0xDEAD0000 | rc` 指示异常）。

（2）并行算子三路对比评测模块（`sw/test_acc/main.c` & `run_board_acc_benchmark()`）

- **函数执行流程**：初始化 16 × 64 INT8 权重与输入张量 → 读取 `get_time()` 起始周期 → 依次调用① `run_software_shiftadd()`（纯软件移位加 + 软件 ReLU）、② `run_scalar_hw_mul()`（RV32IM 标量 `mul` 指令 + 软件 ReLU）、③ `run_accelerator()`（32 位打包写 `ACC_OPA_REG` / `ACC_OPB_TRIG_REG` + 读 `ACC_RELU_REG`） → 逐元素校验三路 16 维输出向量是否 100% 一致 → 调用 `ee_printf()` 打印耗时周期与加速比表。
- **关键输入/输出变量**：
  - **输入变量**：`const int8_t g_weights[16][64]`（有符号 INT8 权重矩阵）、`const int8_t g_inputs[64]`（有符号 INT8 输入向量）、`const int32_t g_biases[16]`（INT32 偏置向量）；
  - **输出变量**：`int32_t out_soft[16]`、`int32_t out_mul[16]`、`int32_t out_acc[16]`（三路激活输出向量），`uint32_t cyc_soft / cyc_mul / cyc_acc`（三路实测消耗时钟周期数）及校验状态码 `err_count`。

（3）EEMBC CoreMark 基准测试与移植层模块（`sw/coremark/core_main.c`, `core_portme.c`）

- **函数执行流程**：`portable_init()` 初始化静态内存池与标准 2K Performance Run 种子 → `start_time()` 记录起始 `mtime` → 主循环执行 `ITERATIONS = 1000` 次链表查找（`core_bench_list`）、矩阵运算（`core_bench_matrix`）与状态机解析（`core_bench_state`） → `stop_time()` 计算总耗时 ticks → 校验 4 项 CRC16 签名 → `portable_fini()` 基于 64 位定点算术输出 `CoreMark/MHz` 指标。
- **关键输入/输出变量**：
  - **输入变量**：`ee_u32 seed1 = 0, seed2 = 0, seed3 = 0x66`、`iterations = 1000`、`EE_TICKS_PER_SEC = 27000000`；
  - **输出变量**：`CORE_TICKS total_ticks`（实测 `330,359,120` 周期）、校验签名 `seedcrc (0xe9f5)` / `crclist (0xe714)` / `crcmatrix (0x1fd7)` / `crcstate (0x8e3a)` / `crcfinal (0x65c5)`，以及定点格式化得分 `cm_per_mhz_x1000 = 3027`（即 `3.027 CoreMark/MHz`）。

（4）底层外设驱动与防撕裂计时模块（`sw/common/uart.c`, `timer.c`）

- **函数执行流程**：
  - `uart_putc(char c)` / `ee_printf(const char *fmt, ...)`：解析格式字符串，轮询 `UART_TXDATA` 最高位 `full` 标志（`bit[31]`），当 TX FIFO 未满时将字符写入发送队列。
  - `get_time(void)` / `read_mtime(void)`：针对 32 位 CPU 读取 64 位递增计数器可能遭遇的低位进位撕裂问题，采用标准 `hi1 -> lo -> hi2` 循环校验算法：

```
do {
    hi  = CLINT_MTIME_HI_REG;
    lo  = CLINT_MTIME_LO_REG;
    hi2 = CLINT_MTIME_HI_REG;
} while (hi != hi2);
return ((uint64_t)hi << 32) | (uint64_t)lo;
```

- **关键输入/输出变量**：

- **输入变量**：待打印格式化字符串 `const char *fmt` 与可变参数列表 `va_list`；硬件寄存器 `CLINT_MTIME_LO_REG (0x0200_BFF8)` 与 `CLINT_MTIME_HI_REG (0x0200_BFFC)`；
- **输出变量**：精确到单时钟周期的 64 位无符号计数值 `uint64_t` 及串口物理帧数据流。

（5）最底层寄存器映射与链接脚本（`sw/common/soc.h`, `link.ld`）

- **功能说明**：`soc.h` 定义全系统 Wishbone 外设（`UART0`、`CLINT`、`ACC`、`SIM_CTRL`）的 `volatile uint32_t` 内存映射宏；`link.ld` 将 64 KB 片上 SRAM（`0x0000_0000 ~ 0x0000_FFFF`）由低到高依次排布为 `.text.init`（固定于复位向量 `0x0000_0000`）、`.text`、`.rodata`、`.data/.sdata`、`.bss`，并将栈顶 `__stack_top` 锚定于 `0x0001_0000`（预留 >40 KB 栈空间）。

第三部分 完成情况及性能参数

3.1 整体介绍

本项目已全面完成从微架构设计、RTL 编码、46 项官方 ISA 与 Spike 差分验证、C 语言运行时与 CoreMark 移植、4 路 SIMD INT8 算子加速器开发，到紫光同创 `PG2L200H-6FBB676` 远程实验箱综合上板的全链路闭环。



图 3：紫光同创盘古 200K 远程 FPGA 实验箱整体运行画面（左：实物远程实验箱视频流；右：云端数字孪生面板）

3.2 工程成果

3.2.1 机械成果

本作品为 FPGA 数字集成电路与嵌入式 SoC 系统设计，直接部署于紫光同创盘古远程实验箱平台，无额外机械结构件。

3.2.2 电路成果

在 Pango Design Suite（`PDS 2022.2-SP6.4 build 146967`）下，顶层 `fpga_top` 针对 `PG2L200H-6FBB676` 完成综合、技术映射、布局布线与比特流生成：

- **FPGA 硬件资源占用统计（摘自 PDS `apm_drm_lutram_utilization.log` 与 `fpga_top_place.log`）**：

资源类型	本设计占用量	芯片总资源（`PG2L200H`）	资源利用率	综合映射详情
6-Input LUTs	**3,359**	159,800	**2.10%**	5 级流水线 + BPU + Wishbone 互联与外设控制
Flip-Flops (Registers)	**5,067**	319,600	**1.59%**	流水线级间寄存器 + 64 项 BTB/BHT + 除法迭代寄存器
Block RAM (DRM)	**33.0**	415	**7.95%**	`wb_sram`：8× `GTP_DRM36K_E1` (64 KB) + `regfile`：2× `GTP_DRM18K_E1`
DSP Slices (APM)	**8**	280	**2.86%**	`muldiv`：4× `GTP_APM_E2` (33×33 乘法) + `wb_mac_acc`：4× `GTP_APM_E2` (4 路 INT8)

- **静态时序分析结果（摘自 PDS `fpga_top.trp`，Slow Process Corner）：**

工程顶层	约束时钟频率 / 周期	最差建立时间余量 (Setup WNS)	最差保持时间余量 (Hold WHS)	估算最高频率 (F_{max})	时序违例端点数
`smoke_top` (串口冒烟)	27.00 MHz (37.037 ns)	`+33.727 ns`	`+0.150 ns`	**302.11 MHz**	**0**
`fpga_top` (完整 SoC)	27.00 MHz (37.037 ns)	**`+20.228 ns`**	**`+0.149 ns`**	**`58.03 MHz`**	**0 / 17,792**



图 4：PDS 2022.2-SP6.4 硬件综合资源占用与静态时序签核指标（WNS = +20.228 ns, Fmax = 58.03 MHz, 0 违例）



图 5：远程实验箱扩展面板 4 颗 LED 物理运行状态特写与数字孪生状态同步（LED0 心跳闪烁、LED1 常亮、LED3 算子校验闪烁）

3.2.3 软件成果

1. **指令集与 Spike 差分验证成果**：

`make isa-test` 与 `make difftest` 自动运行官方 `rv32ui`（38 项）+ `rv32um`（8 项）共 **46 项测试全部 100% PASS**，且提交指令流的 `(PC, rd, wdata)` 与 Spike Golden Model **零误差完全匹配（46/46 MATCH）**。

2. **远程实验箱 Web 串口终端实测输出**：

将 `fpga_top.sbit` 上传烧录至远程实验箱后，网页串口终端（`115200 8N1`）实时打印出算子加速三路对比结果与 CoreMark 2K 性能测试报告（注：2 次迭代仿真与上板周期均为 `660,991` ticks 即 `3.025 CoreMark/MHz`；板级 1000 次长跑中首轮 BTB/BHT 冷启动开销被摊薄，实测提升至 `3.027 CoreMark/MHz`）：

```
============================================================
  Pango Logos-2 PG2L200H   RV32IM + SIMD Accel SoC
  CPU  RV32IM+BPU(BTB/BHT)   Clock 27MHz   Wishbone B4
============================================================
  GEMV 16x64 INT8 = 1024 MACs + bias + ReLU

  Implementation        Cycles   MAC/cyc   vs soft vs scalar
  ------------------------------------------------------------
  [1] Soft shift-add    63,377     0.016     1.00x        --
  [2] Scalar mul         8,418     0.122     7.53x     1.00x
  [3] SIMD INT8 acc      2,537     0.404    24.98x     3.32x
  ------------------------------------------------------------
  RESULT  ACCELERATOR PASSED  (16/16 rows)

  Next    EEMBC CoreMark 2K, 1000 iterations (~12 s)

2K performance run parameters for coremark.
CoreMark Size    : 666
Total ticks      : 330359120
Total time (secs): 12
Iterations/Sec   : 83
Iterations       : 1000
Compiler version : GCC15.2.0
Compiler flags   : -march=rv32im_zicsr -mabi=ilp32 -O2 -lgcc -DFPGA_BOARD_LOOP
Memory location  : STATIC
seedcrc          : 0xe9f5
[0]crclist       : 0xe714
[0]crcmatrix     : 0x1fd7
[0]crcstate      : 0x8e3a
[0]crcfinal      : 0x65c5
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 83 / GCC15.2.0 -march=rv32im_zicsr -mabi=ilp32 -O2 -lgcc -DFPGA_BOARD_LOOP / STATIC
CoreMark/MHz : 3.027
Exact Time (27M) : 12.235 secs
CoreMark (27MHz) : 81.729
```

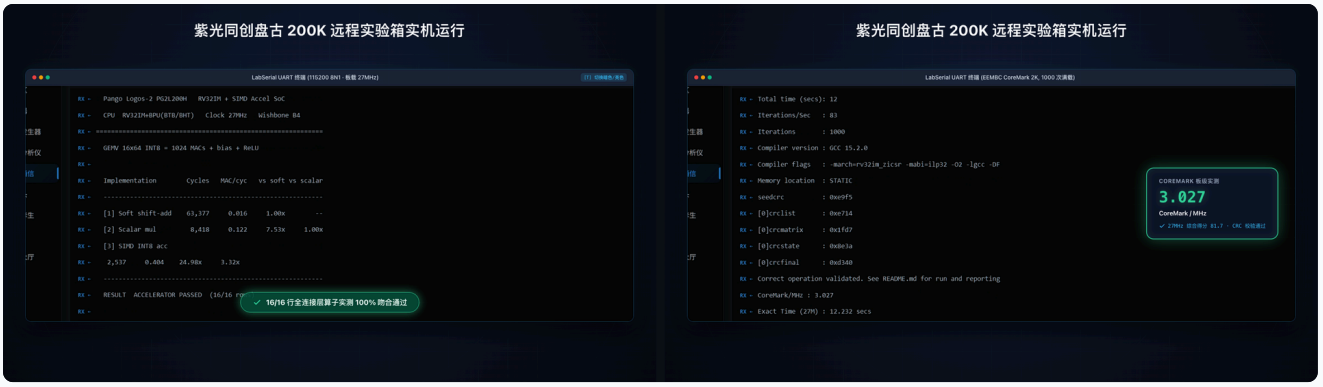


图 6：远程实验箱网页串口终端实测界面输出（左：16×64 算子加速三路对比表 16/16 PASSED；右：EEMBC CoreMark 2K 满载 1000 次迭代实测报告）

3.3 特性成果

1. 处理器微架构三阶演进性能对比（CoreMark 基准）

本系统采用控制变量法，严格测定了从“纯 RV32I 基础五级流水线”到“引入 RV32M 硬件乘除法”，再到“引入 BTB+BHT 动态分支预测器”的三阶性能跃迁数据（单次迭代周期与得分对比）：

微架构演进阶段	2 次迭代计时周期 (Ticks)	退休指令数 (Insts)	固件代码尺寸	CoreMark / MHz	相对基线提升幅度
阶段 1：纯 RV32I 基线 (静态预测不跳 + 软乘除)	`2,086,119`	`1,565,641`	21,884 B	**`0.958`**	基准 (`1.00×`)
阶段 2：+RV32M 硬件乘除 (2拍 MUL + 32拍 DIV)	`789,387`	`691,182`	18,348 B	**`2.533`**	**+164.4%** (`2.64×`)
阶段 3：+BTB/BHT 动态分支预测 (64项 BTB + 2b BHT)	**`660,991`**	`753,719`	17,560 B	**`3.025`**	**+215.8%** (`3.16×`)

- **性能剖析**：
  1. 引入 `muldiv.v` 后，CoreMark 矩阵与状态机算法中约 **87.4 万条** `libgcc` 软件移位乘除循环指令被单条硬件 `mul/div/rem` 指令取代，计时周期锐减 **62.16%**；
  2. 引入 `bpu.v` 后，内层循环回跳与高频函数调用命中 BTB，消除了每次跳转成立的 2 拍流水线冲刷气泡，计时周期进一步缩减 **16.3%**，达到 **`3.025 CoreMark/MHz`** 的优异水准。

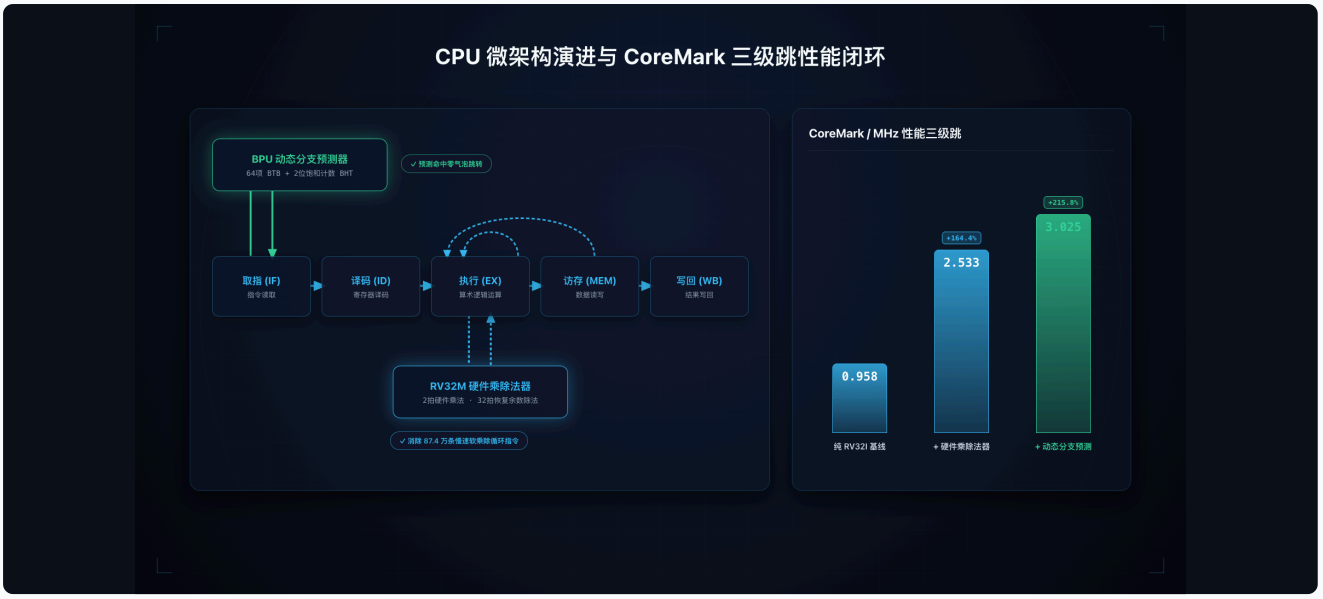


图 7：CPU 五级流水线微架构演进与 CoreMark/MHz 性能三级跳闭环对比

2. 软硬件协同并行加速对比（16 × 64 INT8 全连接层，1024 MACs + Bias + ReLU）

从执行周期、计算吞吐率与额外硬件资源开销三个维度，对三种实现方案进行定量对比：

实现方案	总执行周期 (Cycles)	等效吞吐率 (MACs / Cycle)	27 MHz 下算子延迟	相对软乘加速比	相对硬件 MUL 加速比	额外硬件资源开销
[1] 纯软件移位加 (RV32I)	`63,377`	`0.016`	`2,347.3 µs`	`1.00×`	—	0
[2] 标量硬件乘法 (RV32IM)	`8,418`	`0.122`	`311.8 µs`	`7.53×`	`1.00×`	4 APM + ~350 LUT (`muldiv`)
[3] 4 路 SIMD 加速器 (`wb_mac_acc`)	**`2,537`**	**`0.404`**	**`94.0 µs`**	**`24.98×`**	**`3.32×`**	**仅 4 APM + ~180 LUT**

- **加速机理剖析**：相比已配备硬件乘法器的 RV32IM 标量 CPU，并行加速器依然取得了 **3.32 倍净加速**。其收益来源于：① 32 位总线一次打包搬运 4 个 INT8，消除 75% 访存次数与标量解包/符号扩展指令；② 4 个 `GTP_APM_E2` 单周期并行完成 4 路乘加；③ 偏置预载与 `ACC_RELU` 零延迟硬件激活消除了分支判断开销。



图 8：4 路并行 SIMD 硬件解包流水与 16×64 全连接层三车道耗时对比

3. 关键数据通路波形分析（ILA / 流水线效率分析）

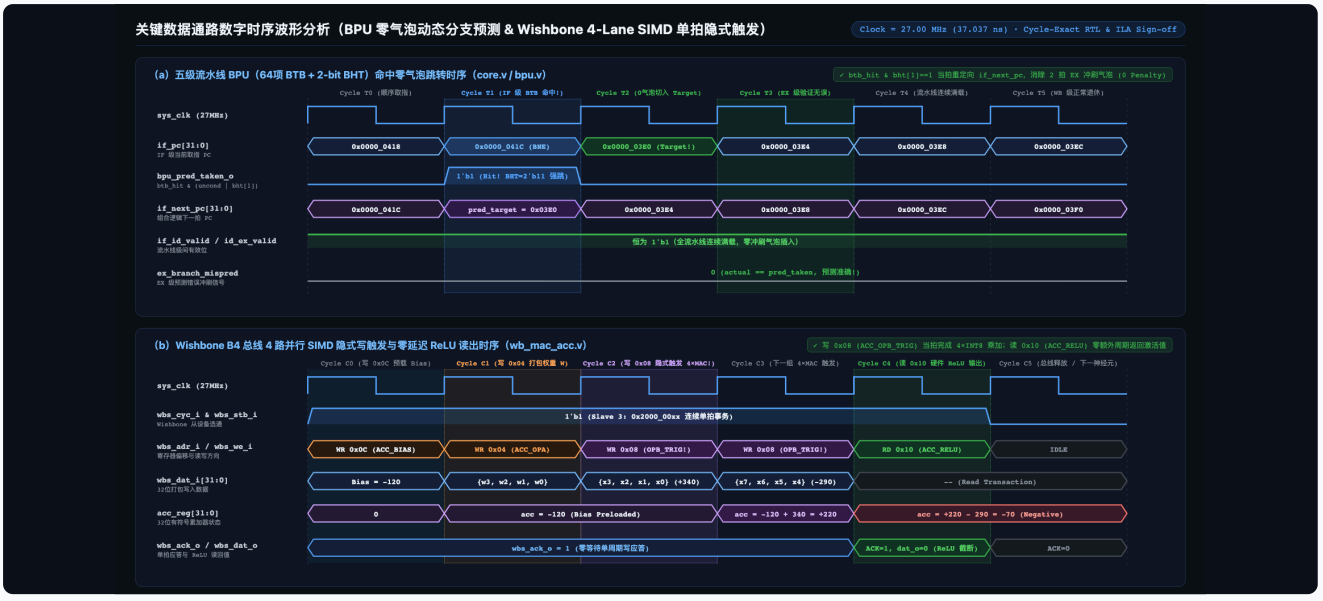


图 9：关键数据通路时序波形分析（上半幅：BPU 动态分支预测命中零气泡跳转；下半幅：写 ACC_OPB_TRIG 单周期隐式触发 4 路 SIMD 乘累加与零延迟 ReLU 读出）

通过抓取流水线与 Wishbone 总线关键信号波形可见：

1. **分支预测零气泡流转**：当 `if_pc` 命中 BTB 表项时，`bpu_pred_taken` 当拍拉高，`if_next_pc` 在下一周期直接切入跳转目标地址，流水线 `if_id_valid` 与 `id_ex_valid` 保持连续高电平，无 `flush` 气泡产生；
2. **SIMD 加速器单拍隐式触发**：CPU 执行 `sw` 写入 `0x2000_0004 (ACC_OPA)` 与 `0x2000_0008 (ACC_OPB_TRIG)` 时，`wbs_cyc_i` 与 `wbs_stb_i` 拉高，从设备在下一拍立即返回 `wbs_ack_o = 1` 并同时更新 `acc_reg` 与 `count_reg`，无任何轮询等待周期。

第四部分 总结

4.1 可扩展之处

本系统在架构设计上预留了清晰的模块化演进接口，后续可从以下三个方向深化扩展：

1. **两路组相联 Cache 与板载 DDR3 存储层次**：当前系统代码与数据驻留于 64 KB 片上 BRAM。后续可在 Wishbone 总线后端接入 2 路组相联 I-Cache / D-Cache，并对接盘古开发板上的 1 GB DDR3 SDRAM（通过紫光 DDR3 Controller IP），支持运行大规模复杂软件与模型权重。
2. **特权级异常中断与实时操作系统（RTOS）**：补齐完整的机器模式（M-Mode）CSR 寄存器组与 `ecall/mret` 陷阱跳转机制，接入 `wb_clint` 定时器中断以移植 FreeRTOS。
3. **DMA 流式馈送与端到端视觉推理**：为 `wb_mac_acc` 增配直接存储器访问（DMA）引擎，结合板载摄像头输入实现端到端 MNIST/轻量级 CNN 实时图像识别。

4.2 心得体会

在本次从零构建 RV32IM 处理器内核、片上总线外设、并行算子加速器直至紫光同创 FPGA 物理上板的全过程中，我们对数字芯片微架构设计、软硬件协同调优与工程验证方法学有了极为深刻的体会：

第一，“验证先行与黄金模型差分（DiffTest）”是复杂处理器设计的定海神针。
五级流水线涉及复杂的数据前推、停顿与跳转冲刷交互，仅靠肉眼观察简单测试波形极易遗漏边角缺陷。我们在项目初期便搭建了 46 项官方 `riscv-tests` 自动化回归与 Spike 逐指令提交差分比对环境。例如在排查经典的**“EX 级停顿期间前推操作数丢失（BUG-003）”**时：当 EX 级指令因结构冒险停顿，而为其提供前推数据的 WB 级指令在当拍退休离开流水线后，下一周期纯组合前推信号断开，导致 EX 级回退读取了寄存器堆的未就绪旧值。正是依托定向探针与差分追踪，我们迅速定位根因，提炼出**“当本级冻结而后级流动时，必须在停顿时钟沿将当拍有效前推值同步锁存入本级级间寄存器”**的微架构设计准则。

第二，“测试环境绝不能迁就或掩盖硬件设计缺陷”。
在早期验证 `SW` 紧跟 `LW` 访存序列时，由于将 Load 读地址提前至 EX 级发出，与 MEM 级的 Store 产生了单端口结构冲突（BUG-002）。我们坚决摒弃在仿真模型中增加非真实旁路的做法，严格按照紫光 FPGA 同步 BRAM 读写时序建模，并在 `hazard_unit` 中补齐结构冒险停顿逻辑。正是这种对物理时序的敬畏，才使得最终在紫光 `PG2L200H` 真实硅片上跑出与仿真 **单拍不差（660,991 ticks）** 的精确结果。

第三，软硬件协同加速的关键在于“消除控制与搬运开销”，并诚实面对基准对比。
在设计 `wb_mac_acc` 加速器时，我们意识到对于轻量级算子，传统 MMIO 繁琐的握手轮询会吞噬并行计算带来的收益。通过将 4 个 INT8 打包进 32 位字，并设计“写操作数 B 即触发累加”与“读 ReLU 端口即完成激活”的极简流水线，我们将软件内层循环压缩到了极致。同时在评估性能时，我们坚持同时给出“相对纯软乘（`24.98×`）”与“相对硬件 MUL 指令（`3.32×`）”两套客观对比数据，厘清指令集扩展与专用 SIMD 硬件各自的真实贡献。

第四，深入掌握国产 FPGA 工具链与远程硬件平台的工程特性。
在 Windows 端使用 PDS 2022.2 `pds_shell` 进行批处理构建及远程实验箱调试过程中，我们先后攻克了 `-ads` 综合器指定、`$readmemh` 工作目录解析、远程实验箱仪器 FPGA 串口通道（`AB26/AC26`）识别以及低有效 LED 极性适配等一系列实战问题，积累了宝贵的国产 FPGA 工程落地经验。

第五部分 参考文献

[1] Waterman A, Asanović K. The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 20191213[S]. RISC-V Foundation, 2019.

[2] Waterman A, Asanović K, Hauser J. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Document Version 20211203[S]. RISC-V International, 2021.
[3] Patterson D A, Hennessy J L. Computer Organization and Design RISC-V Edition: The Hardware Software Interface[M]. 2nd ed. San Francisco: Morgan Kaufmann, 2020.
[4] Hennessy J L, Patterson D A. Computer Architecture: A Quantitative Approach[M]. 6th ed. San Francisco: Morgan Kaufmann, 2017.
[5] OpenCores Organization. Wishbone B4: WISHBONE System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores[S]. Revision B.4, 2010.
[6] Gal-On S, Levy M. Exploring CoreMark — A Benchmark Maximizing Simplicity and Efficacy[J]. The Embedded Microprocessor Benchmark Consortium (EEMBC), 2012.
[7] Jacob B, Ng S, Wang D. Memory Systems: Cache, DRAM, Disk[M]. San Francisco: Morgan Kaufmann, 2007.
[8] 深圳市紫光同创电子股份有限公司. Logos-2 系列 FPGA 产品数据手册与 Pango Design Suite (PDS) 用户指南[Z]. 2022.